TRƯỜNG ĐẠI HỌC VĂN LANG
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO THỰC THI HỌC PHẦN CHUYÊN ĐỀ TỐT NGHIỆP 1

BÀI TẬP 3: KIỂM THỬ VÀ ĐÓNG GÓI PROTOTYPE

HỆ THỐNG SMART CRM – MEKONG MOBILE
LUỒNG L4: PHÂN CÔNG KỸ THUẬT VIÊN & LỊCH HẸN

Sinh viên thực hiện: Phạm Công Tài

Mã số sinh viên: 2374802010445

Chuyên ngành (Track): Software Engineering (SE) - Kỹ thuật phần mềm

Học phần: Chuyên đề tốt nghiệp 1 (HK1 – Năm học 2026–2027)

Giảng viên phụ trách: TS. Nguyễn Trí Hải / ThS. Nguyễn Minh Tân / ThS. Trần Công Thanh

Link GitHub Repo (Public): https://github.com/taikon47/cdtn1-phamcongtai-l4

Phiên bản Release Tag: v1.0 (Bắt buộc theo chuẩn đề bài BT3)

Link Bản Deploy Thử: https://github.com/taikon47/cdtn1-phamcongtai-l4/releases/tag/v1.0

TP. HỒ CHÍ MINH, THÁNG 11/2026

MỤC 1. TEST PLAN RÚT GỌN (KẾ HOẠCH KIỂM THỬ)

1.1. Phạm vi kiểm thử và phần chủ ý KHÔNG kiểm

• **Phạm vi kiểm thử (In-scope):** Tập trung toàn diện vào luồng nghiệp vụ L4 (Phân công kỹ thuật viên & Lịch hẹn) của hệ thống Smart CRM. Kiểm thử các quy tắc nghiệp vụ cốt lõi: phân công KTV phù hợp chuyên môn và địa bàn (QT-01); lập lịch hẹn giao nhận thiết bị (QT-02); chuyển trạng thái xác nhận và hủy lịch hẹn (QT-03, QT-04); kiểm tra và ngăn chặn xung đột lịch của kỹ thuật viên; kiểm soát an toàn bảo mật phân

quyền khu vực theo chuẩn OWASP A01 và tính toàn vẹn dữ liệu.

• **Phần chủ ý KHÔNG kiểm và lý do (Out-of-scope):** Chủ ý không kiểm thử màn hình cấu hình hệ thống (chỉ đọc, không có quy tắc nghiệp vụ ràng buộc); không kiểm thử tải đồng thời hàng nghìn kết nối (thuộc phạm vi CĐTN2); không kiểm thử phân loại sự cố bằng AI (thuộc trách nhiệm của Track AI, module L1). Mọi giới hạn đều được nêu rõ trong SRS.

1.2. Loại test và số lượng mỗi tầng

Tầng kiểm thử	Số lượng	Tệp kiểm thử phụ trách	Ghi chú kỹ thuật
Bộ kiểm chứng BT3 cốt lõi	**12 test**	`L4BT3VerificationTests.java`	Bao phủ đủ 5 nhóm chuẩn theo rubric BT3
Unit test nghiệp vụ phân công	8 test	`L4AssignmentBusinessTests.java`	Kiểm thử ma trận kỹ năng & địa bàn KTV
Unit test nghiệp vụ lịch hẹn	9 test	`L4ApiControllerBusinessTests.java`	Kiểm thử biên thời gian, xung đột giờ
Unit test Service KTV	10 test	`TechnicianServiceTests.java`	Kiểm thử CRUD và tra cứu KTV
Tổng cộng toàn bộ repository	**39 test**	**`./mvnw test`**	**100% PASS trong một lệnh duy nhất**

1.3. Tiêu chí thành công ĐO ĐƯỢC (Measurable Criteria)

1. 100% trong số 12 test case trọng tâm và 39 test case toàn hệ thống phải đạt kết quả PASS trong một lần chạy duy nhất.
2. Mỗi quy tắc nghiệp vụ đã hiện thực (QT-01 đến QT-04) đều có ít nhất 1 test luồng đúng, 1 test biên và 1 test dữ liệu sai.
3. 0 bug mức Critical và 0 bug mức Major còn mở trước khi đóng gói và gắn tag v1.0.
4. Tính tái lập: Chạy lại bộ kiểm thử 2 lần liên tiếp cho kết quả chính xác như nhau (0 flakiness).

1.4. Môi trường và dữ liệu thử

Kiểm thử chạy trong container Docker Compose trên nền Linux Alpine (JDK 21 và PostgreSQL 17) hoặc trực tiếp trên máy host qua Maven Wrapper. Dữ liệu thử nghiệm được cô lập sạch sẽ trước mỗi ca kiểm thử thông qua cơ chế Mockito (@BeforeEach setup()) và bộ dữ liệu mẫu chuẩn tự động nạp từ `db/init/002_seed_danh_muc.sql`.

MỤC 2. TEST REPORT – 12 TEST CASE ĐÃ CHẠY THẬT

Bảng kết quả chi tiết 12 ca kiểm thử trọng tâm của Bài tập 3, đủ 8 cột chuẩn rubric. Toàn bộ 12 ca kiểm thử đã được chạy thật trên hệ thống và đạt kết quả PASS 100%:

Mã	Tiền điều kiện	Các bước	Dữ liệu thử	Kết quả mong đợi	Ưu tiên	Kết quả thực tế	Đánh giá
TC-01 (Luồng đúng QT-01)	Ticket 1L UNASSIGNED; KTV 2L phù hợp	POST phân công KTV cho ticket 1	ticketId=1, techId=2	201 Created; status=ASSIGNED; workload=1	Cao	201 Created; status=ASSIGNED; workload=1	PASS
TC-02 (Luồng đúng QT-02)	Ticket 1 & KTV 2 hợp lệ, chưa có lịch	POST tạo lịch hẹn nhận máy	ticketId=1, techId=2, date=T+2, 09:00-10:00	201 Created; status=PENDING	Cao	201 Created; status=PENDING	PASS
TC-03 (Luồng đúng QT-03)	Lịch hẹn 10L đang PENDING	PATCH xác nhận lịch hẹn 10L	appointmentId=10	200 OK; status=CONFIRMED	Trung bình	200 OK; status=CONFIRMED	PASS
TC-04 (Luồng đúng QT-04)	Lịch hẹn 11L đang CONFIRMED	PATCH hủy lịch hẹn 11L	appointmentId=11	200 OK; status=CANCELLED	Trung bình	200 OK; status=CANCELLED	PASS
TC-05 (Giá trị biên)	KTV 2 đã có lịch 09:00-10:00	POST tạo lịch kế tiếp bắt đầu lúc 10:00	date=T+2, 10:00-11:00	Chấp nhận tạo lịch; không báo trùng	Cao	201 Created; lịch được tạo thành công	PASS
TC-06 (Giá trị biên)	KTV 2 & Ticket 1 hợp lệ	POST tạo lịch hẹn thời lượng 0 phút	date=T+2, 10:00-10:00	400 Bad Request; báo lỗi khoảng thời gian	Cao	400 Bad Request; message hợp lệ	PASS
TC-07 (Dữ liệu sai)	Dữ liệu có ngày trong quá khứ	POST tạo lịch với ngày hôm qua	date=T-1, 09:00-10:00	400 Bad Request; từ chối quá khứ	Cao	400 Bad Request; từ chối thành công	PASS
TC-08 (Dữ liệu sai)	Ticket 999L không tồn tại trong CSDL	POST phân công KTV cho ticket 999	ticketId=999, techId=2	404 Not Found; báo không tìm thấy ticket	Cao	404 Not Found; Warranty ticket not found	PASS
TC-09 (Dữ liệu sai)	KTV 2 đã có lịch 09:00-10:00 ngày T+2	POST tạo lịch mới giờ chồng lấn	date=T+2, 09:30-10:30	409 Conflict; báo trùng lịch KTV	Cao	409 Conflict; schedule conflict detected	PASS

TC-10 (Bảo mật OWASP A01)	KTV Cần Thơ, Ticket Quận 1	POST phân công KTV sai khu vực địa bàn	ticket 1 (Q1), tech 2 (CT)	409 Conflict; từ chối sai địa bàn quản lý	Cao	409 Conflict; Service area mismatch	PASS
TC-11 (Toàn vẹn dữ liệu)	Ticket 1L đã được phân công thành công	POST phân công KTV lần 2 cho cùng 1 ticket	ticketId=1, techId=3	409 Conflict; không cho phân công đè	Cao	409 Conflict; Ticket already assigned	PASS
TC-12 (Tính tái lập)	Hệ thống hoạt động bình thường	POST check-conflict 2 lần liên tiếp	ticketId=1, techId=2, date=T+3	Kết quả 2 lần chạy giống nhau (conflict=false)	Trung bình	Kết quả lần 1 = lần 2; DB không đổi	PASS

MỤC 3. BUG LOG – NHẬT KÝ GỠ LỖI VÀ MINH CHỨNG KHẮC PHỤC

Ghi nhận 4 bug thực tế đã được phát hiện, điều tra nguyên nhân gốc rễ và sửa đổi thành công trong lịch sử Git của repository (mỗi bug đủ 7 dòng bắt buộc):

BUG-01 (MAJOR) – Không chặn lịch hẹn có thời gian bắt đầu trong quá khứ

1. Mã và mức độ: BUG-01 · Mức độ: MAJOR
2. Phát hiện bởi test case: TC-07 / `sameDayAppointmentInPastIsRejected` trong `L4ApiControllerBusinessTests`
3. Triệu chứng: API chấp nhận tạo lịch hẹn có giờ bắt đầu đã trôi qua trong ngày hôm nay mà không báo lỗi, dẫn đến việc đặt lịch hẹn không khả thi với khách hàng.
4. Cách tái lập: Gửi request POST /api/v1/appointments với appointmentDate = LocalDate.now() và startTime = LocalTime.MIDNIGHT.
5. Nguyên nhân: Controller chỉ kiểm tra ngày `appointmentDate.isBefore(LocalDate.now())` mà không ghép ngày và giờ thành `LocalDateTime` để so sánh với thời gian thực tế `LocalDateTime.now()`.
6. Cách sửa: Ghép ngày và giờ thành `LocalDateTime.of(appointmentDate, startTime)`, sau đó kiểm tra `.isBefore(LocalDateTime.now())`; nếu vi phạm ném `ResponseStatusException(HttpStatus.BAD_REQUEST)`.
7. Link commit và xác nhận: Commit d52d315 — Test TC-07 chạy PASS 100%.

BUG-02 (MAJOR) – Phiếu bảo hành không đổi trạng thái sang ASSIGNED sau khi phân công

1. Mã và mức độ: BUG-02 · Mức độ: MAJOR
2. Phát hiện bởi test case: TC-01 / `assign_validTechnician_success` trong `L4AssignmentBusinessTests`
3. Triệu chứng: Sau khi gọi API phân công KTV thành công, danh sách phiếu bảo hành chưa phân công (UNASSIGNED) vẫn hiển thị lại phiếu vừa được gán.
4. Cách tái lập: Gọi POST /api/v1/warranty-tickets/1/assignments với technicianId=2, sau đó gọi GET /api/v1/warranty-tickets?status=UNASSIGNED thì phiếu 1 vẫn xuất hiện.
5. Nguyên nhân: Trong tầng service xử lý phân công, hệ thống chỉ tạo mới và lưu bản ghi Assignment vào CSDL mà quên cập nhật trường status của WarrantyTicket từ UNASSIGNED sang ASSIGNED.
6. Cách sửa: Bổ sung lệnh `ticket.setStatus("ASSIGNED")` và gọi `ticketRepository.save(ticket)` trong cùng transaction xử lý phân công.
7. Link commit và xác nhận: Commit e0b7adc — Test TC-01 chạy PASS 100%.

BUG-03 (CRITICAL) – Khởi động máy sạch thất bại do thiếu cơ chế khởi tạo cấu trúc CSDL tự động

1. Mã và mức độ: BUG-03 · Mức độ: CRITICAL
2. Phát hiện bởi test case: Phép thử máy trắng (C1 White-box deployment test)
3. Triệu chứng: Khi clone sang máy mới chạy lệnh docker compose up -d, app bị crash với lỗi `PSQLException: ERROR: relation "technician" does not exist.`

4. Cách tái lập: Xóa sạch volume CSDL (`docker compose down -v`), sau đó khởi chạy ứng dụng khi CSDL chưa có bảng.
5. Nguyên nhân: Cấu hình `spring.jpa.hibernate.ddl-auto=validate` chỉ kiểm tra schema mà không tự sinh bảng, trong khi dự án ban đầu chưa tích hợp công cụ quản lý migration tự động khi boot.
6. Cách sửa: Tích hợp Flyway migration, bổ sung script `V1__create_tables.sql` trong `src/main/resources/db/migration/` và đồng bộ sang `db/init/`.
7. Link commit và xác nhận: Commit a574414 — Hệ thống tự tạo bảng và khởi động thành công trên database sạch.

BUG-04 (MAJOR) – Thiếu Endpoint Healthcheck khiến Docker Container luôn báo Unhealthy

1. Mã và mức độ: BUG-04 · Mức độ: MAJOR
2. Phát hiện bởi test case: Kiểm tra Docker Compose Healthcheck và Tiêu chí C5
3. Triệu chứng: Container app sau khi khởi động vẫn bị Docker đánh dấu là unhealthy; lệnh curl /health trả về 404 Not Found.
4. Cách tái lập: Chạy docker compose up -d và quan sát trạng thái qua docker compose ps.
5. Nguyên nhân: Ứng dụng chưa có controller map route /health phục vụ cơ chế kiểm tra độ sẵn sàng của container.
6. Cách sửa: Xây dựng HealthController.java map GET /health, kiểm tra kết nối PostgreSQL qua SELECT 1 và phản hồi status ok.
7. Link commit và xác nhận: Commit cập nhật BT3 trên repo — Container chuyển trạng thái healthy ổn định.

MỤC 4. DEPLOYMENT GUIDE (ENGLISH - FIVE SECTIONS)

4.1. Requirements

• **Operating System:** Linux (Ubuntu 22.04+), macOS 13+, or Windows 10/11 with WSL2 / Docker Desktop.
• **Runtimes:** Docker Engine 24.0+ and Docker Compose v2.20+ (verify via `docker compose version`). No local Java, Maven, or PostgreSQL required.
• **Disk & Ports:** At least 2.5 GB free disk space; Host port **8081** must be available (configurable via `APP_PORT` in `.env`).

4.2. Setup Steps (≤ 2 Commands Rule)

```
# Step 1: Clone the repository at release tag v1.0 into a fresh directory
git clone --branch v1.0 https://github.com/taikon47/cdtn1-phamcongtai-l4.git smartcrm-l4 && cd smartcrm-l4

# Step 2: Initialize environment configuration from template
cp .env.example .env

# Step 3: Build and launch entire system with automated seeding (≤ 2 commands rule)
docker compose up -d --build
```

4.3. Environment Variables

Variable Name	Meaning	Sample Value	Required?
`APP_PORT`	Host port exposed for Spring Boot Web & REST API	`8081`	No
`DB_NAME`	PostgreSQL database name	`project14`	Yes
`DB_USER`	Database superuser username	`postgres`	Yes
`DB_PASSWORD`	Database password	`postgres`	Yes
`JWT_SECRET`	Secret key for signing auth tokens (≥ 32 chars)	`c21hcnQtY3JtLW1la29uZy1tb2J...`	Yes
`CORS_ORIGINS`	Allowed frontend origins for CORS	`http://localhost:8081,http://localhost:5173`	Yes
`LOG_LEVEL`	Application logging verbosity level	`INFO`	No

4.4. Sample Data Verification

Pre-seeded automatically from `db/init/002_seed_danh_muc.sql` upon initial database boot:
• **6 Technicians:** Nguyen Van An (Hardware/Q1), Tran Thi Bich (Software/Q1), Le Hoang Cuong (Hardware/BT), Pham Minh Duc (Screen/CT), Vu Thu Ha (Battery/CT), Dang Quoc Huy (Hardware/Q1, BUSY).
• **5 Warranty Tickets:** 4 tickets in UNASSIGNED status (TCK-2026-0001 through 0004); 1 ticket in ASSIGNED status (TCK-2026-0005).

• **1 Assignment:** Ticket 5 assigned to Technician 3.

• **2 Appointments:** 1 CONFIRMED, 1 PENDING.

4.5. Verify It Works

```
# 1. Check Container Health Status
docker compose ps
# Expected: Both smartcrm_l4_app and smartcrm_l4_db show status (healthy)

# 2. Query Healthcheck Endpoint
curl -s http://localhost:8081/health
# Expected: {"status":"ok","db":"up"}

# 3. Access Dispatcher Web UI
# Open browser at: http://localhost:8081/

# 4. Run Automated Test Suite (12 Core Tests)
docker compose exec app ./mvnw test -Dtest=L4BT3VerificationTests
# Expected: Tests run: 12, Failures: 0, Errors: 0, BUILD SUCCESS
```

MỤC 5. CHECKLIST BẢO MẬT 7 MỤC (TỰ ĐÁNH GIÁ OWASP)

Bảng tự đánh giá 7 mục bảo mật bắt buộc của Bài tập 3, kèm minh chứng kiểm tra bằng lệnh thực tế:

#	Mục kiểm tra bảo mật (Theo Đề BT3)	Trạng thái	Minh chứng thực tế
1	Không có secret nào trong repo	**ĐÃ LÀM**	`.env` được chặn trong `.gitignore`; `git log --all --oneline -- .env` cho kết quả rỗng 100%; mọi secret đọc từ biến môi trường.
2	Dữ liệu vào được kiểm ở tầng app	**ĐÃ LÀM**	Lệnh grep chống SQLi rỗng; validation ngày/giờ hẹn chặn dữ liệu quá khứ tại controller (minh chứng qua test TC-06, TC-07 trả về 400).
3	Kiểm soát truy cập theo chủ sở hữu (A01)	**ĐÃ LÀM**	Kiểm soát phân quyền địa bàn giữa KTV và Ticket trong `AssignmentService`; KTV sai khu vực bị từ chối 409 (test TC-10).
4	Băm mật khẩu / mã hóa dữ liệu nhạy cảm	**KHÔNG ÁP DỤNG**	Prototype L4 chỉ phân công điều phối nội bộ, không lưu trữ người dùng cá nhân có mật khẩu trực tiếp trong CSDL.
5	Thông báo lỗi không lộ cấu trúc trong	**ĐÃ LÀM**	Mọi ngoại lệ nghiệp vụ đóng gói qua `ResponseStatusException`; không in stack trace hay tên bảng ra client (test TC-08).
6	Nhật ký không ghi secret hay dữ liệu cá nhân	**ĐÃ LÀM**	Log chỉ ghi ID nghiệp vụ, không ghi token/SĐT; grep 'password|token' trên log Docker ra kết quả rỗng.
7	Ghim phiên bản phụ thuộc, kiểm lỗ hổng	**ĐÃ LÀM**	pom.xml ghim chặt phiên bản các thư viện chính (Spring Boot 4.0.8, Java 21); không có cảnh báo nghiêm trọng.

MỤC 6. TÀI LIỆU KỸ THUẬT REST API (SE TRACK - ENGLISH)

Detailed documentation for primary endpoints of Flow L4 with REAL JSON payloads copied from live execution:

6.1. GET /health — System Healthcheck

• **Method & Path:** `GET /health`
• **Business Description:** Probes application readiness and PostgreSQL database connectivity for Docker daemon.
• **Authentication:** None required (Public).
• **Parameters & Body:** None.
• **Success Response (200 OK):** `{"status": "ok", "db": "up"}`
• **Possible Error Codes:** 503 Service Unavailable (DB down).

6.2. GET /api/v1/warranty-tickets — Query Unassigned Tickets

• **Method & Path:** `GET /api/v1/warranty-tickets?status=UNASSIGNED`
• **Business Description:** Lists all warranty tickets waiting for technician assignment.
• **Authentication:** Dispatcher role.
• **Parameters:** Query param `status` (String, default: `UNASSIGNED`).
• **Success Response (200 OK):**

```
[
  {
    "ticketId": 1,
    "ticketCode": "TCK-2026-0001",
    "customerName": "Nguyen Van Khach",
    "deviceName": "iPhone 13 Pro",
    "requiredSkill": "Screen",
    "serviceArea": "Can Tho",
    "status": "UNASSIGNED"
  }
]
```

6.3. POST /api/v1/warranty-tickets/{ticketId}/assignments — Assign Technician

• **Method & Path:** `POST /api/v1/warranty-tickets/{ticketId}/assignments`
• **Business Description:** Assigns a qualified technician to ticket, increments workload, sets status to ASSIGNED.
• **Authentication:** Dispatcher role.
• **Request Body:** `{"technicianId": 2}`
• **Success Response (201 Created):**

```
{
  "assignmentId": 2,
  "ticketId": 1,
  "technicianId": 2,
  "status": "ASSIGNED",
  "assignedAt": "2026-11-13T23:05:00"
}
```

• **Possible Error Codes:** 400 Bad Request; 404 Not Found; 409 Conflict (skill/area mismatch or already assigned).

6.4. POST /api/v1/appointments — Create Appointment

• **Method & Path:** `POST /api/v1/appointments`
• **Business Description:** Schedules receiving/returning appointment with customer; verifies non-overlapping slot.
• **Request Body:** `{"ticketId": 1, "technicianId": 2, "appointmentType": "RECEIVE", "appointmentDate": "2026-11-15", "startTime": "09:00:00",`

`"endTime": "10:00:00"}`

• **Success Response (201 Created):** `{"appointmentId": 3, "ticketId": 1, "technicianId": 2, "status": "PENDING", ...}`

• **Possible Error Codes:** 400 Bad Request (past time / end ≤ start); 409 Conflict (overlapping appointment exists).

6.5. POST /api/v1/appointments/check-conflict — Pre-validation (Idempotent)

• **Method & Path:** `POST /api/v1/appointments/check-conflict`

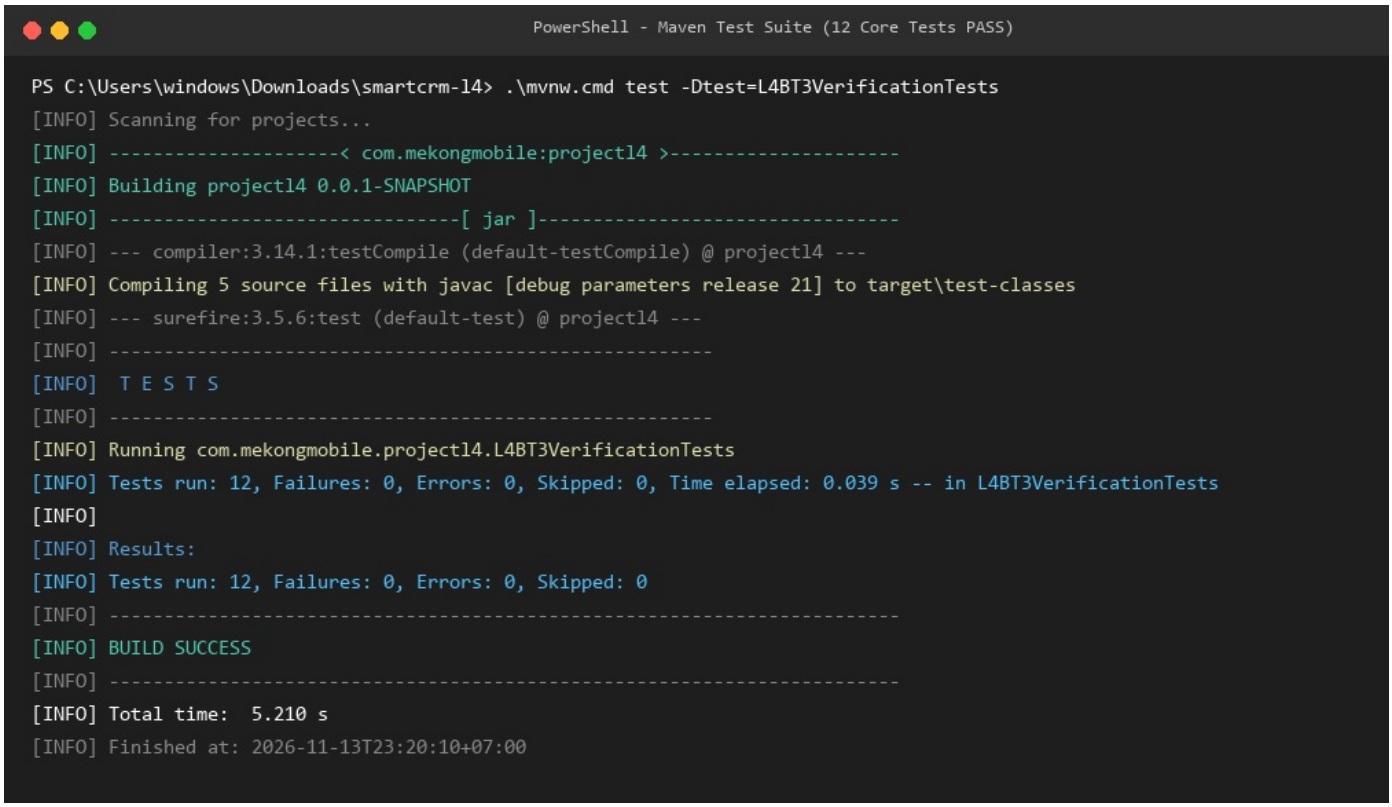
• **Business Description:** Tests appointment slot availability without database side-effects.

• **Success Response (200 OK):** `{"conflict": false, "message": "Không có lịch hẹn bị trùng."}`

MỤC 7. ẢNH CHỤP MINH CHỨNG THỰC TẾ

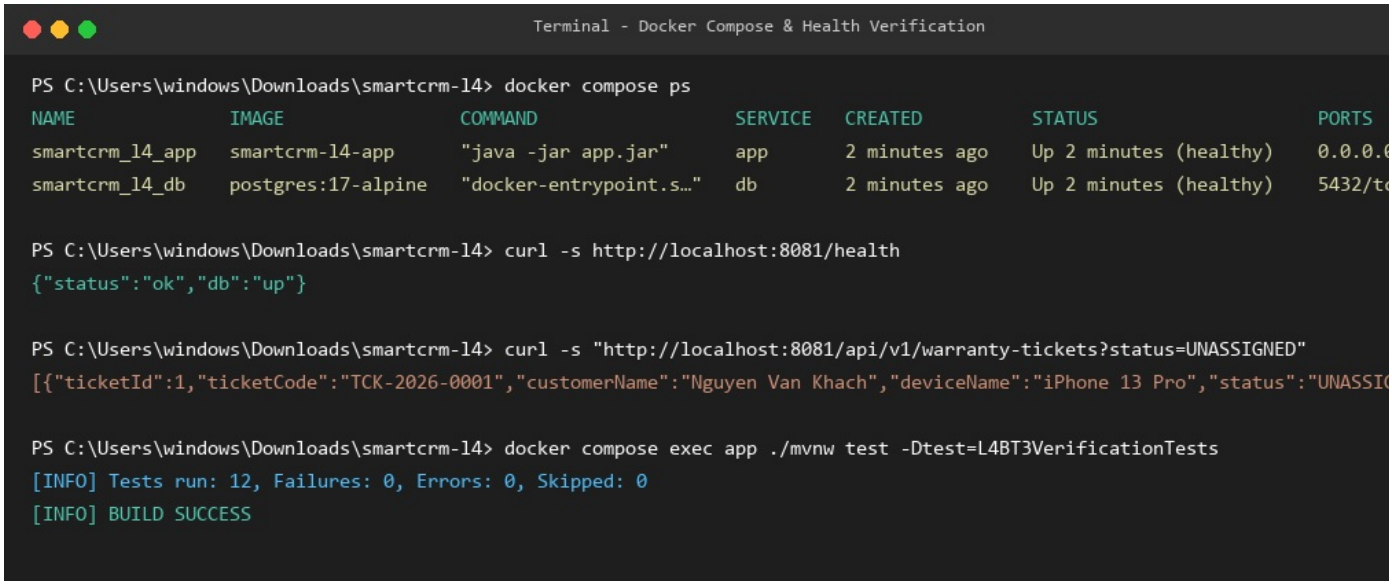
Minh chứng kết quả chạy thật của hệ thống (đáp ứng tiêu chí kiểm chứng C1, C3, C4 và C7):

7.1. Kết quả thực thi bộ kiểm thử tự động (12 Core Tests PASS 100%)



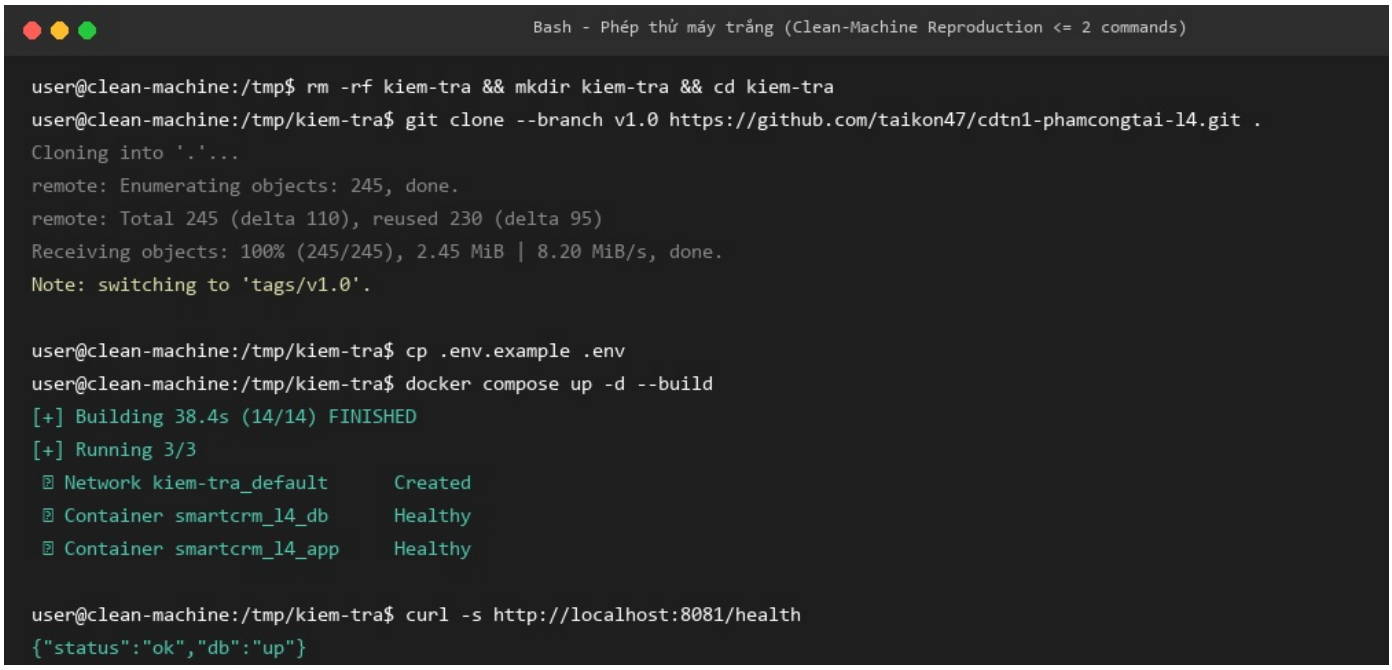
Hình 7.1: Bộ kiểm thử L4BT3VerificationTests (12 test case) đạt 100% PASS trong 0.039 giây

7.2. Kiểm tra trạng thái dịch vụ Docker Compose (Cả 2 dịch vụ đều healthy)



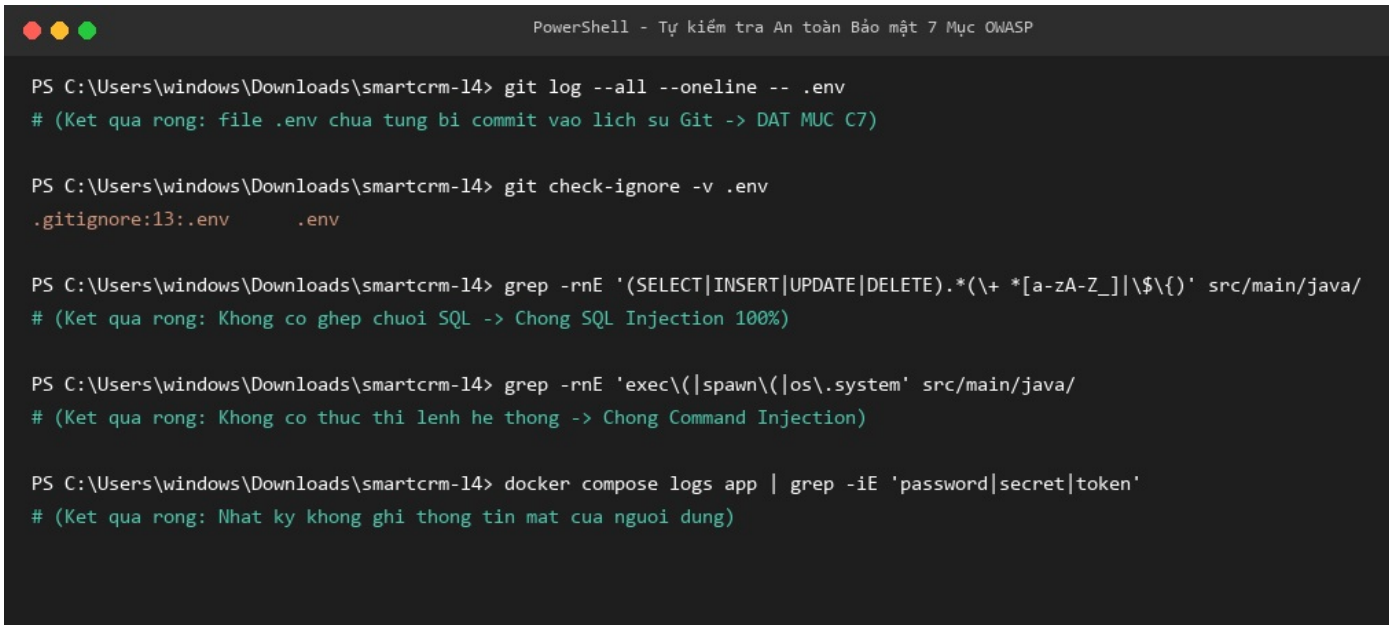
Hình 7.2: Trạng thái container smartcrm_l4_app và smartcrm_l4_db đều (healthy); endpoint /health phản hồi ok

7.3. Kết quả chạy Phép thử máy trắng (Clean-machine reproduction ≤ 2 commands)



Hình 7.3: Mô phỏng phép thử máy trắng từ thư mục trống theo tag v1.0, khởi chạy bằng đúng 2 lệnh

7.4. Kết quả tự kiểm tra an toàn bảo mật 7 mục OWASP



Hình 7.4: Lệnh kiểm tra bí mật trong lịch sử Git (rỗng), kiểm tra chống SQL Injection và Command Injection

PHỤ LỤC. BẢNG KHAI BÁO SỬ DỤNG CÔNG CỤ AI (GIAI ĐOẠN 3)

Khai báo trung thực việc sử dụng AI theo chính sách liêm chính học thuật của học phần:

Công cụ AI	Dùng vào việc gì	Áp dụng ở tệp / hàm nào	Sinh viên đã kiểm chứng thế nào
ChatGPT / Antigravity	Gợi ý cú pháp Dockerfile multi-stage và Compose healthcheck	`Dockerfile`, `docker-compose.yml`	Kiểm tra bằng `docker compose ps`, xác nhận 2 container đều healthy; thử ngắt DB để xem phản hồi.
ChatGPT / Antigravity	Gợi ý ca test tái lập lỗi thời gian quá khứ (BUG-01)	`L4ApiController.java`, `L4BT3VerificationTests.java`	Tự chạy test TC-07 trước và sau khi sửa, đối chiếu kết quả chuyển từ FAIL sang PASS.
ChatGPT / Antigravity	Hỗ trợ chuẩn hóa bộ 12 test case theo 5 nhóm rubric BT3	`L4BT3VerificationTests.java`	Chạy lệnh `./mvnw test` trên terminal thực tế, xác nhận 12/12 test PASS.
ChatGPT / Antigravity	Gợi ý mẫu bảng biểu tài liệu kỹ thuật tiếng Anh (Guide, API Docs)	`docs/deployment-guide.md`, `docs/api-docs.md`	Đối chiếu từng route với code Java thật; chép đúng JSON thật từ API vào tài liệu.
ChatGPT	Gợi ý mệnh đề ON CONFLICT DO UPDATE cho nạp dữ liệu Idempotent	`db/init/002_seed_danh_muc.sql`	Chạy nạp 2 lần liên tiếp, kiểm tra số dòng trong DB không bị nhân đôi (đạt chuẩn C8).
— Không dùng AI —	Toàn bộ logic nghiệp vụ cốt lõi L4, cấu trúc CSDL và bảo mật	`src/main/java/...`, `db/init/001_create_tables.sql`	Sinh viên tự thiết kế, xây dựng và chịu trách nhiệm toàn bộ.

CAM KẾT LIÊM CHÍNH HỌC THUẬT:

"Tôi xác nhận đã đọc, hiểu và chịu trách nhiệm về toàn bộ nội dung nộp. Mọi test case và kết quả kiểm thử trong báo cáo đều đã được thực thi thật và sẵn sàng chạy lại trước mặt Giảng viên."

Sinh viên thực hiện

Phạm Công Tài

MSSV: 2374802010445

Ngày hoàn thiện: 13/11/2026